

# Frontend Engineering Assignment

---

## Overview

You are provided with the design of a user-facing web application. Your task is to build a production-ready **Content Management System (CMS)** and integrate it with the provided frontend so that all content displayed on the website is managed dynamically through the admin panel. You are encouraged to make your own architectural decisions and clearly document any assumptions you make.

---

## Objective

Design and develop an admin panel that enables authenticated administrators to manage the content displayed throughout the provided website.

The public-facing website should consume content through your APIs rather than relying on hardcoded data.

---

**Figma Design Link:** [\[LINK\]](#)

## Preferred Technology Stack

While you are free to choose technologies where appropriate, the preferred stack is:

|                       |                               |
|-----------------------|-------------------------------|
| <b>Frontend</b>       | <b>Next js, Redux toolkit</b> |
| <b>Backend</b>        | <b>Express js</b>             |
| <b>Infrastructure</b> | <b>Docker</b>                 |
| <b>Database</b>       | <b>As per your choice</b>     |

## Functional Requirements

### Authentication

Implement an authentication system for administrators. At a minimum, the application should support:

- Login
- Logout

The implementation details are entirely your choice.

## **Admin Dashboard**

Create an admin dashboard that serves as the entry point into the CMS.

The dashboard should provide a good administrative experience, but its design and functionality are left to your discretion.

---

## **Content Management**

The CMS should allow administrators to manage the content displayed throughout the provided frontend.

---

## **Rich Content Support**

Not all website content consists of simple titles and descriptions. Your solution should consider content such as:

- Long-form text
- Multiple paragraphs
- Lists
- Nested lists
- Tables
- Mathematical equations
- Structured documentation
- Mixed content

You are free to decide how this content is stored, edited, and rendered.

The goal is not to implement every possible feature, but to demonstrate thoughtful engineering decisions that allow the system to evolve over time.

## **Public Website Integration**

The public frontend should retrieve its content from your backend rather than relying on static data.

## **State Management**

Use Redux Toolkit where appropriate.

How much application state belongs in Redux versus local component state is part of the design decision we are evaluating.

---

## **Responsive Design**

The admin panel should function well across different screen sizes.

---

## **Technical Expectations**

There is intentionally no required project structure. You are expected to organize the application in a way that you believe would scale as the project grows. Similarly, there are no required API conventions or database models. Your implementation should reflect how you would approach a real production application.

## **Documentation**

Include a README describing:

- Setup instructions
- Technology choices
- Architecture overview
- Assumptions
- How to run the project

## **Deliverables**

Please submit:

- Source code
- GitHub repository
- README
- Environment variable template
- Any sample credentials or seed data required to evaluate the application

## **Submission**

Please provide:

- GitHub repository link
- Setup instructions
- Any credentials required for evaluation
- A brief explanation of the architectural decisions you made